# Quick Start

The example code in this quick start guide is provided for educational and demonstration purposes only. It may not represent best practices for production use. This quick start was last updated for **UAI.LiteRTLM v2.1.0-preview.5**.

## Aim

UAI.LiteRTLM aims to provide a 1:1 interface to the LiteRT-LM multiplatform C API. Since LiteRT-LM's C API is highly unstable and not intended for public use, this section explains the breaking changes policy and LiteRT-LM version support for UAI.LiteRTLM.

## Versioning Scheme

This package tries to follow semantic versioning, with a few caveats:

- **All** versions leading up to the stable release of the LiteRT-LM C API **will** be *preview* versions and are not recommended for production use.
    - Example: `2.1.0-preview.3`, `2.0.0-preview.1`
- `MINOR` version updates (`x.Y.z`, where `Y` is the `MINOR` version number) indicate a change in the LiteRT-LM tag and/or commit ID used by the package.
    - Example: UAI.LiteRTLM `2.1.0-preview.x` -> LiteRT-LM `v0.15.0-alpha0` (`ad53ed1`), `2.0.0-preview.x` -> LiteRT-LM `v0.14.0` (`80f301f`)
- `MINOR` version updates are ***very likely*** to contain breaking changes due to changes in the LiteRT-LM C API.
- `PATCH` version updates are represented by the `preview` label (`x.y.0-preview.Z`, where `Z` is the `PATCH` version number) instead of the standard `x.y.Z` format.
- `PATCH` version updates aim to be backwards compatible, with exceptions for application-level metadata changes that may be required by the update.
    - Example: `2.1.0-preview.2` switches from WebGPU to OpenCL acceleration on Android, requiring additional entries in the application's `AndroidManifest.xml`.

## Prebuilt Binaries

While UAI.LiteRTLM aims to support all platforms supported by LiteRT-LM, it currently only includes prebuilt native binaries for a subset of them, depending on the package version. The package aims to provide LiteRT-LM binaries without C patches and builds the default `c:litert-lm` Bazel target with platform-specific command-line arguments. The build scripts for all platforms are available in `Native/`. You can build your own binaries for additional

platforms and include them in your project's `Plugins` directory. UAI.LiteRTLM should use them as-is for that platform.

UAI.LiteRTLM also does not include every LiteRT-LM GPU accelerator. Only one accelerator is included per platform.

Do not include your own binaries for platforms or accelerators that UAI.LiteRTLM already provides binaries for, as this can cause conflicts. If you want to use your own binaries instead, customize the package and remove the included ones.

Build script contributions are always welcome!

# API

The package API is roughly split into two parts: the low-level API and the high-level API.

## Low-level API (`NativeAPI`)

The low-level API, exposed by the `NativeAPI` static class, is a simple wrapper around all functions exposed by the LiteRT-LM C API. It works directly with pointers and `MonoPInvokeCallback`s, so using this API requires you to manage native C memory yourself.

## High-level API (`ManagedAPI`)

The high-level API, defined in `ManagedAPI.cs`, is a collection of handle classes that wrap all native structures defined by the native API and implement `IDisposable`. All pointers and native callbacks are handled automatically, and some wrappers also provide additional abstractions to make them easier to use. These wrapper classes also implicitly convert to `IntPtr`, allowing you to use them with the native API if needed.

Currently, only one feature implemented by the native API is **not** exposed by the managed API. You cannot pass custom additional data to streamed operations because the wrappers use their own data to track managed `StreamCallback`s.

# Version - LiteRT-LM - Platforms - Accelerators Table

| UAI.LiteRTLM | LiteRT-LM | Included Platforms | Included Accelerators |
|---|---|---|---|
| 2.2.0-preview.1+ | v0.15.0 (`2117fc4`**) | Android (arm64)<br>macOS (arm64)<br>iOS (arm64, sim_arm64)<br>Windows (x64) | CPU<br>OpenCL (Android)<br>Metal (macOS, iOS)<br>WebGPU (Windows) |

| UAI.LiteRTLM | LiteRT-LM | Included Platforms | Included Accelerators |
|---|---|---|---|
| 2.1.0-preview.5+ | v0.15.0-alpha0 (`ad53ed1`) | Android (arm64)<br>macOS (arm64)<br>iOS (arm64, sim_arm64)<br>Windows (x64) | CPU<br>OpenCL (Android)<br>Metal (macOS, iOS*)<br>WebGPU (Windows) |
| 2.1.0-preview.4 | v0.15.0-alpha0 (`ad53ed1`) | Android (arm64)<br>macOS (arm64)<br>iOS (arm64, sim_arm64) | CPU<br>OpenCL (Android)<br>Metal (macOS, iOS*) |
| 2.1.0-preview.3 | v0.15.0-alpha0 (`ad53ed1`) | Android (arm64)<br>macOS (arm64) | CPU<br>OpenCL (Android)<br>Metal (macOS) |
| 2.1.0-preview.2 | v0.15.0-alpha0 (`ad53ed1`) | Android (arm64) | CPU<br>OpenCL |
| 2.1.0-preview.1 | v0.15.0-alpha0 (`ad53ed1`) | Android (arm64) | CPU<br>WebGPU |
| 2.0.0-preview.1 | v0.14.0 (`80f301f`) | Android (arm64) | CPU<br>WebGPU |

* Complete Metal acceleration is available on iOS devices, but this LiteRT-LM version does not provide a Metal-accelerated `TopKSampler` for iOS simulators.
** LiteRT-LM v0.15.0 GPU sampling is bugged on Android (arm64) and Windows, so this release contains a patched prebuilt dependency (`libLiteRtTopKOpenClSampler.so`) from commit `8bee4dd`. There is no patch for Windows as the latest upstream prebuilt still has the issue.

# Android GPU Acceleration

To enable GPU acceleration on Android, add the following to the `<application>` section of your `AndroidManifest.xml` to enable OpenCL:

```
<uses-native-library android:name="libvndksupport.so" android:required="false"/>
<uses-native-library android:name="libOpenCL.so" android:required="false"/>
```

# Example Script

Documentation for this package is still a WIP. The following example demonstrates a streamed single-turn conversation using the high-level API, including benchmark collection.

```
using Uralstech.UAI.LiteRTLM;

private async Awaitable RunConversation()
{
    LiteRTLMNativeLogging.SetMinLogLevel(LogSeverity.Info);

    string modelPath = Path.Join(Application.persistentDataPath, "model.litertlm");
    string cacheDir = Path.Join(Application.temporaryCachePath, "modelCache");
    Directory.CreateDirectory(cacheDir);

    using EngineSettings engineSettings = new(modelPath, BackendNames.GPU);
    engineSettings.SetEnableSpeculativeDecoding(true); // Can perform badly on
WebGPU on Windows.
    engineSettings.SetCacheDir(cacheDir);
    engineSettings.EnableBenchmark();

    using Engine engine = new(engineSettings);
    using ThinkingConfig thinkingConfig = new();
    thinkingConfig.SetEnableThinking(false);

    using ConversationConfig conversationConfig = new();
    conversationConfig.SetThinkingConfig(thinkingConfig);

    using Conversation conversation = new(engine, conversationConfig);
    Debug.Log("Engine and conversation created.");

    const string message = "{\"role\":\"user\",\"content\":\"What is the tallest
building in the world?\"}";
    TaskCompletionSource<bool> completionSource = new();

    void OnChunk(StreamChunk chunk)
    {
        Debug.Log("Got chunk:"
                  + $"\n\tText: {chunk.GetText()}"
                  + $"\n\tIsFinal: {chunk.IsFinal()}"
                  + $"\n\tError: {chunk.GetError()}"
        );

        if (chunk.IsFinal())
            completionSource.TrySetResult(true);
```

```
    }

    int result = conversation.SendMessageStream(OnChunk, message);
    Debug.Log("Message sent: " + (result == 0));

    if (result == 0)
        await completionSource.Task;

    using BenchmarkInfo? benchmarkInfo = conversation.GetBenchmarkInfo();
    if (benchmarkInfo == null)
    {
        Debug.Log("No benchmark info found.");
        return;
    }

    BenchmarkInfo.Turn[] prefillTurns = benchmarkInfo.GetPrefillTurns();
    BenchmarkInfo.Turn[] decodeTurns = benchmarkInfo.GetDecodeTurns();

    Debug.Log("Benchmark info:"
              + $"\n\tInitialization time: {benchmarkInfo.GetTotalInitTime()}"
              + $"\n\tTime to first token: {benchmarkInfo.GetTimeToFirstToken()}"
              + $"\n\tPrefill tokens per second: {prefillTurns[0].TokensPerSecond},
for total: {prefillTurns[0].TokenCount} tokens"
              + $"\n\tDecode tokens per second: {decodeTurns[0].TokensPerSecond},
for total: {decodeTurns[0].TokenCount} tokens"
    );
}
```